

适用于 UGUI 的超级滚动视图



目录

1.	包概述 .	3
1.1	列表视图 ..	..3
1.2	网格视图.	..3
1.3	交错视图 .	4
1.4	特殊网格视图	.4
1.5	响应式视图	.5
1.6	聊天视图 .	..5
1.7	画廊	5
1.8	树形视图.	.5
1.9	页面浏览量.	.5
1.10	旋转视图	.5
1.11	嵌套视图 .	.6
1.12	列表视图动画	..6
1.13	可拖动视图 .	..6
2.	循环列表视图 2.	..7
2.1	LoopListView2 概览 ..	..☒->..☒
2.2	LoopListView2 检查器设置 .	..7
2.3	LoopListView2 重要公共方法 .	.11
3.	循环网格视图 .	15
3.1	LoopGridView 概述 .	5
3.2	循环网格视图检查器设置 ..	16
3.3	LoopGridView 重要公共方法 ..	18
4.	Loop交错网格视图.	22
4.1	LoopStaggeredGridView 概述 .	22
4.2	循环交错网格视图 检查器 设置	.23
4.3	LoopStaggeredGridView 重要公共方法 ..	.24

1. 套餐概述

在 UGUI SuperScrollView 包中，有三个主要组件：LoopListView2、LoopGridView 和 LoopStaggeredGridView。

LoopListView2、LoopGridView 和 LoopStaggeredGridView 是附加到 UGUI ScrollRect 相同游戏对象的组件。它们帮助 UGUI ScrollRect 实现高性能且节省内存地支持任意数量的项目。

此软件包包含 88 个示例，这些示例使用了以下三个组件中的一个。

1.1 列表视图

列表视图使用 LoopListView2 组件。

- 1) 列表视图从上到下演示
- 2) 从左到右的列表视图演示
- 3) 列表视图自底向上演示
- 4) 列表视图从右向左演示
- 5) 列表视图选择删除演示
- 6) 列表视图下拉刷新演示
- 7) 列表视图下拉加载更多演示
- 4) 列表视图点击加载更多演示
- 9) 列表视图筛选演示
- 10) 列表视图展开演示
- 11) 列表视图内容适配器演示
- 12) 列表视图多预制件自上而下排列演示
- 13) 列表视图多预制件从左到右排列演示
- 14) 列表视图简单自上而下演示
- 15) 列表视图简单从左到右演示
- 16) ListView 简单加载更多演示
- 17) 列表视图从上到下简单循环演示
- 18) 列表视图简单循环从左到右演示

1.2 网格视图

网格视图使用 LoopGridView 组件。

- 1) 网格视图 从上到下演示

- 2) 网格视图从左到右演示
- 3) GridView 自底向上滚动演示
- 4) 网格视图从右向左演示
- 5) 网格视图点击加载更多演示
- 6) 网格视图选择删除演示
- 7) 网格视图 对角线 左上角演示
- 4) 网格视图对角线右下角演示
- 9) 网格视图对角线选择删除演示
- 10) 网格视图 多预制件 自上而下排列演示
- 11) 网格视图多预制件从左到右排列演示
- 12) 简单的从上到下的 GridView 演示
- 13) 网格视图从左到右简单演示
- 14) 简单的 GridView 过滤器演示
- 15) 网格视图简单对角线右上角演示
- 16) 网格视图简单左下角对角线演示

1.3 错落视图

交错视图使用 `LoopStaggeredGridView` 组件。交错视图是一种 `GridView`，其中的项目在垂直的 `GridView` 中具有不同的高度，在水平的 `GridView` 中具有不同的宽度，例如“www.pinterest.com”网站的样式。

- 1) 从上到下的交错视图演示
- 2) 从左到右交错视图演示
- 3) 交错视图自底向上演示
- 4) 从右向左排列的交错视图演示
- 5) 交错视图 简单自上而下演示
- 6) 交错视图简单从左到右演示
- 7) 交错视图移动到项目演示

1.4 特殊网格视图

特殊网格视图使用 `LoopListView2` 组件。

- 1) 特殊网格视图 从上到下演示
- 2) 特殊网格视图从左到右演示
- 3) 特殊网格视图 选择 删除 演示
- 4) 特殊网格视图下拉刷新演示
- 5) 特殊网格视图上拉加载更多演示
- 6) 特殊网格视图功能自上而下演示
- 7) 从左到右演示的特殊网格视图功能
- 4) `SpecialGridView` 简单自上而下演示
- 9) `SpecialGridView` 简单从左到右演示

1.5 响应式视图

响应式视图使用 LoopListView2 组件。

响应式视图演示

2) 响应式视图刷新加载演示

1.6 聊天视图

聊天视图使用 LoopListView2 组件。

1) 聊天视图演示

2) 聊天视图更改视口高度演示

1.7 图库

画廊使用 LoopListView2 组件。

1) 图库横向演示

2) 图库垂直布局演示

1.8 树状视图

TreeView 使用 LoopListView2 组件。

树形视图演示

2) 带有子项缩进的树形视图演示

3) 带有固定表头的树形视图演示

4) 树形视图简单演示

5) 树视图添加与移除演示

1.9 页面浏览量

PageView 使用 LooplistView2 组件。

1) 页面浏览量演示

2) 页面浏览简单示例

1.10 SpinView

Spin View 使用 LoopListView2 组件。

- 1 旋转日期选择器演示
- 2) SpinTime 选择器演示
- 3) Spin 日期时间选择器演示

1.11 嵌套视图

嵌套视图使用 LoopListView2 或 LoopGridView 组件。

- 1) 嵌套自顶向下演示
- 2) 嵌套从左到右演示
- 3) 从上到下的嵌套网格视图演示
- 4) 从左到右的嵌套网格视图演示
- 5) 嵌套简单列表视图演示
- 6) 嵌套简单网格视图演示
- 7) 嵌套简单特殊网格视图演示

1.12 列表视图动画

列表视图动画使用 LoopListView2 组件。

- 列表视图添加剪辑动画演示
- 2) 列表视图添加淡入淡出动画演示
 - 3) 列表视图添加淡入淡出动画演示
 - 4) 列表视图添加向左滑动动画演示
 - 5) 列表视图向右滑动动画演示
 - 6) 列表视图删除剪辑动画演示
 - 7) 列表视图删除淡出动画演示
 - 4) 列表视图删除剪辑淡入淡出动画演示
 - 1) 列表视图向左滑动删除动画演示
 - 10) 列表视图向右滑动删除动画演示
 - 11) 列表视图展开剪辑动画演示
 - 12) 列表视图展开淡入淡出动画演示
 - 13) 列表视图展开剪裁渐隐动画演示

1.13 可拖动视图

可拖动视图使用 LoopListView2 组件。

可拖动视图：从顶部渐隐至底部演示

2) 可拖动视图淡出左至右演示

3) 可拖动视图从上到下演示

4) 可拖动视图从左向右演示

2. 循环列表视图 2

2.1 LoopListView2 概述

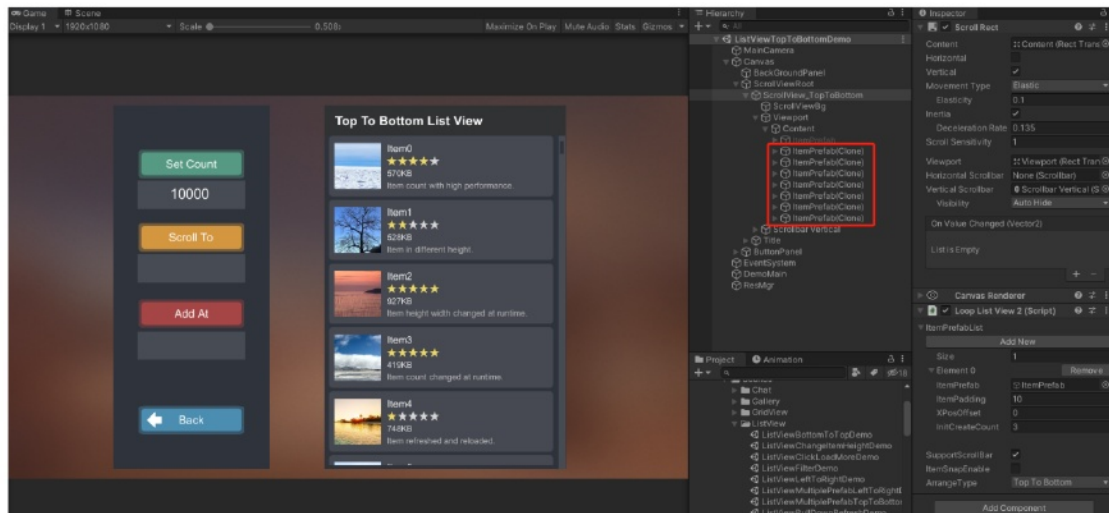
对于包含 10,000 个项目的 ScrollRect，LooplistView2 并不会真正创建 10,000 个项目，而是根据视口的大小创建少量项目。

当滚动视图向上滚动时，LoopListView2 组件会检查最顶部项目的所在位置，一旦最顶部项目移出视口，LoopListView2 组件就会回收最顶部项目。

同时，它会检查最底部项目的所在位置，一旦最底部项目接近视口底部，LooplistView2 组件就会调用 onGetitemByindex 处理程序来创建一个新项目，然后将新创建的项目置于最底部项目的下方，这样新创建的项目就成为了新的最底部项目。

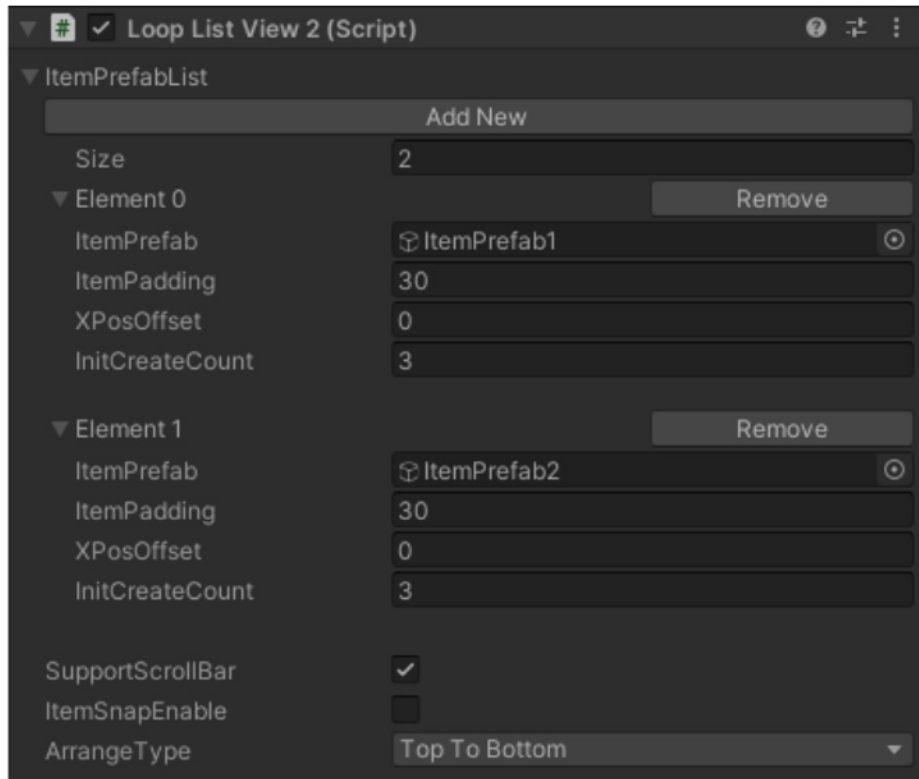
每个项目都可以使用不同的预制件，并且可以有不同的高度/宽度和内边距。

滚动视图（ScrollRect）中有 10000 个项目，但实际上只创建了 7 个。下面的图展示的是从上到下排列的滚动视图（ScrollRect）的样子：



2.2 LooplistView2 检查器设置

在“检查器”中，为了确保 LooplistView2 组件正常工作，有若干参数需要设置。



1) 物品预制列表

现有的要克隆的项目。

每个项目都可以使用不同的预制件，每个预制件都可以有不同的默认内边距（即在 ScrollRect 中每个项目之间的间距大小）。

每个预制件都有不同的默认 x 局部位置（对于垂直滚动视图）或默认 y 局部位置（对于水平滚动视图），这里称为（X/Y）位置偏移。

每个预制件都有一个用于获取和回收操作的池，而 InitCreateCount 则是在启动时池中创建的数量。

实际上，在运行时，每个项目都可以有不同的内边距和（X/Y）位置偏移量，您可以在 onGetItemByIndex 回调中更改项目的内边距和（X/Y）位置偏移量。您可以通过 LooplistViewItem2.Padding 和 LooplistViewItem2.StartPosOffset 来获取/设置它们。

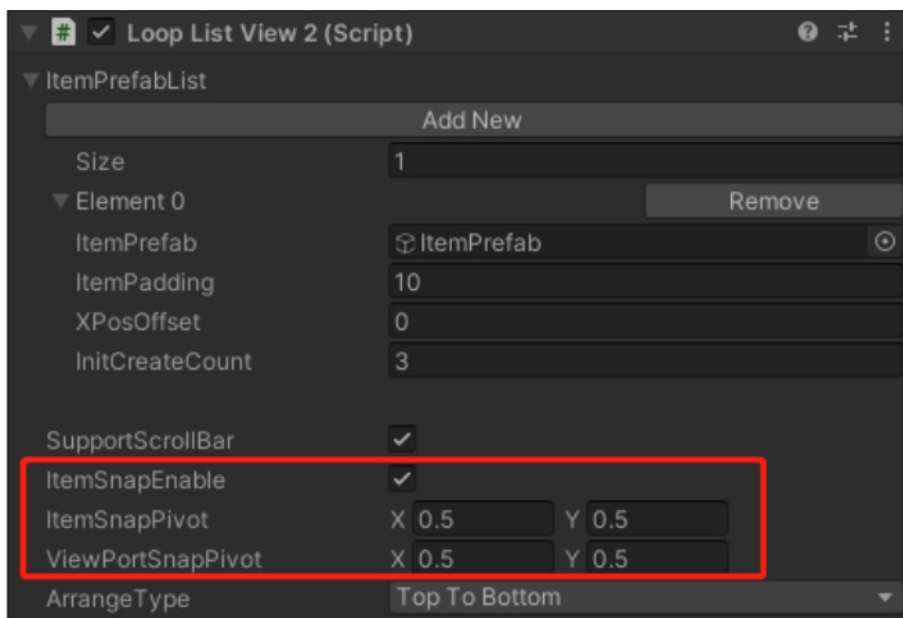
重要提示：所有“ItemPrefab”的本地缩放比例都需要为（1,1,1）。

2) 支持滚动条

如果选中，LoopListView2 组件将支持滚动条。

3) ItemSnapEnable：项快启用

如果选中此选项，LoopListView2 组件将尝试将项目对齐到视口中的配置位置。



4) ItemSnapPivot

这是项目快照枢轴点的位置，以矩形自身大小的分数形式定义。（0,0）对应于左下角，而（1,1）对应于右上角。

5) 视口快照枢轴

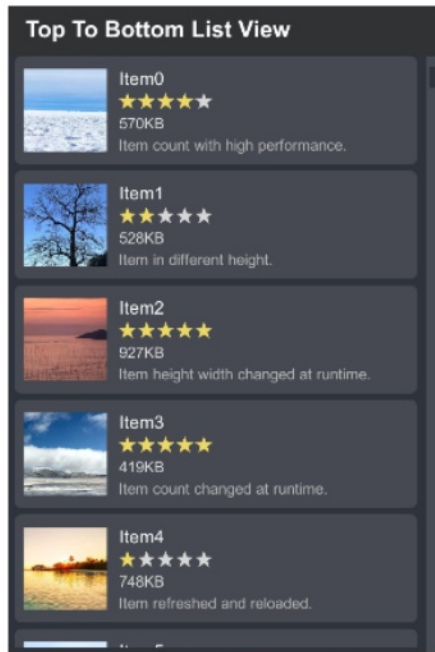
这是 ScrollRect 视口的快照枢轴点的位置，以矩形自身大小的分数形式定义。（0,0）对应于左下角，而（1,1）对应于右上角。

6) 安排类型

这是滚动方向，有四种类型：

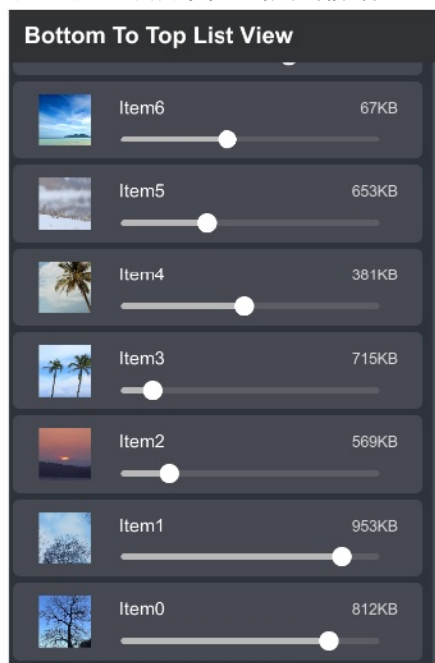
自上而下

此类型用于垂直的滚动视图（ScrollRect），其中 item0、item1……itemN 在滚动视图的视口内从上到下依次排列。



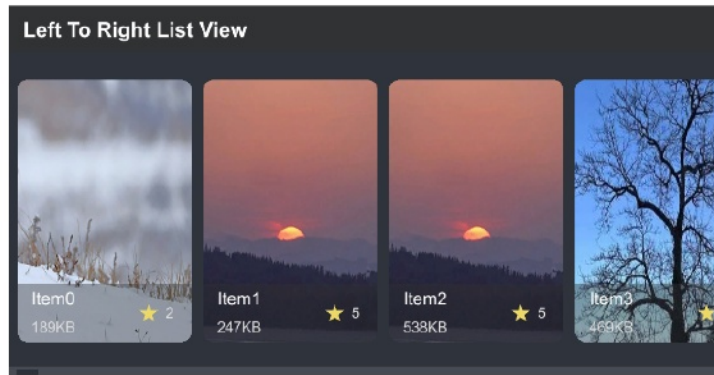
自下而上

这种类型用于垂直的滚动视图（ScrollRect），其中 item0、item1……itemN 在滚动视图的视口中**从下往上依次排列**。



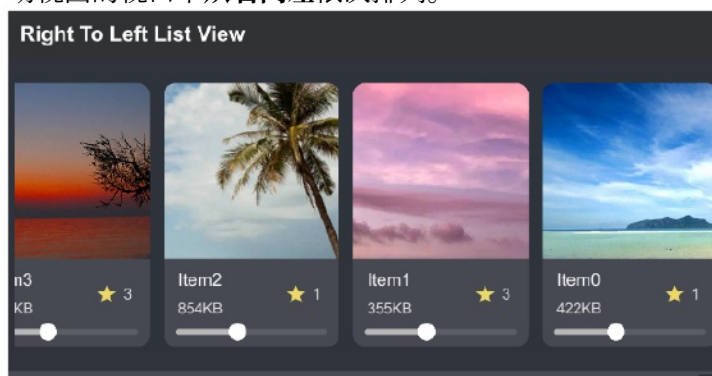
从左到右

此类型用于水平方向的滚动视图（ScrollRect），其中 item0、item1……itemN 在滚动视图的视口内**从左至右依次排列**。



从右到左

此类型用于水平方向的滚动视图（ScrollRect），其中 item0、item1……itemN 在滚动视图的视口中从右向左依次排列。



2.3 LoopListView2 重要公共方法

1) 初始化列表视图方法

```
public void InitListView(int itemTotalCount,
    System.Func<LoopListView2, int, LoopListViewItem2> onGetItemByIndex,
    LoopListViewInitParam initParam = null,
    System.Func<int, (float, float)> onGetItemSizeByIndex = null)
```

此方法可用于初始化 LoopListView2 组件。有 4 个参数：

项目总数

滚动视图中的项目总数。如果此参数设置为 -1，则表示项目数量无限，不支持滚动条，并且项目索引可以是 - 最大整数到 + 最大整数。如果此参数设置为大于等于 0 的值，则项目索引只能从 0 到 **itemTotalCount - 1**。

☒ 根据索引获取项目

当某个项目进入 ScrollRect 视口时，此操作将被调用，并将项目的索引作为参数传递，以便您创建该项目并更新其内容。

初始化参数

有一些参数可以通过 initParam 进行设置。其中非常有用的一个参数是 LoopListViewInitParam.mItemDefaultWithPaddingSize。

当向 ListView 中添加新项时，LoopListView2 在新添加的项显示在视口之前无法得知其实际大小，因此 LoopListView2 会使用带有内边距的默认项大小（默认为 100）作为该项的大小，您可以通过传递自定义的 LoopListViewInitParam 来设置项的默认大小。

例如，

`LoopListViewInitParam` 初始化参数 `initParam` 为默认参数的副本；`initParam.mItemDefaultWithPaddingSize` 设置为 200（此处设置为您项目的默认大小）。`mLoopListView` 初始化列表视图，传入项目总数、通过索引获取项目的回调函数 `OnGetItemByIndex` 以及初始化参数 `initParam`。

获取索引对应项大小的回调函数

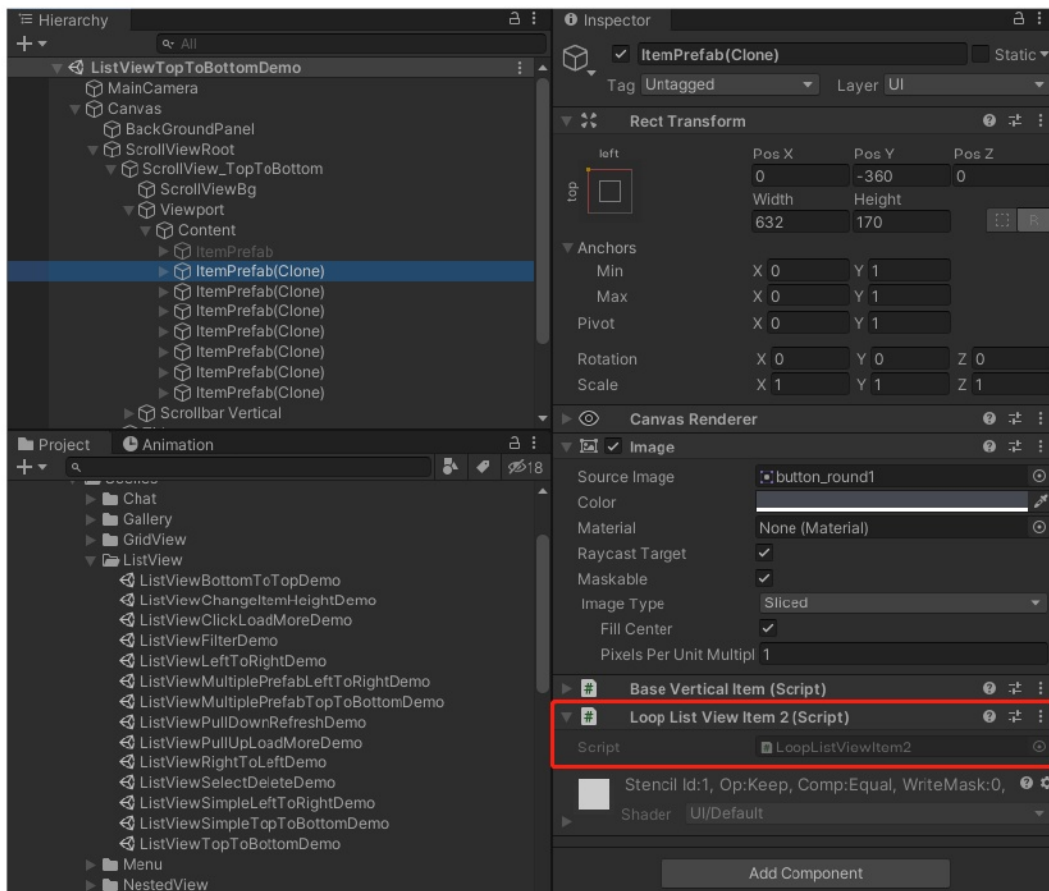
如果您能事先知晓每个项目的大小，您可以设置此委托，该委托用于返回给定项目索引的大小和内边距。目前，此委托主要用于“UpdateItemSizeAtOnce”方法。

例如，当您向 ListView 中添加新项时，如果新项的大小与 LoopListViewInitParam.mItemDefaultWithPaddingSize 不同，并且新项未显示在视口中，那么滚动条的大小将无法正确更新。此时，您可以调用 LoopListView2.UpdateItemSizeAtOnce (newItemIndex, newItemIndex)，让 LoopListView2 调用您的 **onGetItemSizeByIndex** 函数来获取新添加项的实际大小。

循环列表视图项 2

onGetItemByIndex 的返回值。

每个创建的项目都会自动附加一个 LoopListViewItem2 组件。



```
public class LoopListViewItem2 : MonoBehaviour
{
    int mItemIndex = -1;
    int mItemId = -1;
    LoopListView2 mParentListView = null;
    bool mIsInitHandlerCalled = false;
    string mItemPrefabName;
    RectTransform mCachedRectTransform;
    float mPadding;
}
```

mItemIndex 属性表示该项在列表中的索引。

如果 **itemTotalCount** 被设置为 -1, 则 **mItemIndex** 可以从 -MaxInt 到 +MaxInt。

如果 **itemTotalCount** 被设置为大于等于 0 的值, 则 **mItemIndex** 只能从 0 到 **itemTotalCount - 1**。

mItemId 属性表示用户定义的项目 ID。此属性可设置为任何用户定义的整数值, 用于任何目的, 例如帮助查找项目。

以下代码是 **onGetItemByIndex** 的一个示例:

```
LoopListViewItem2 OnGetItemByIndex(LoopListView2 listView, int index)
{
    if (index < 0 || index >= mDataSourceMgr.TotalItemCount)
    {
        return null;
    }
    //get the data to showing
    ItemData itemData = mDataSourceMgr.GetItemDataByIndex(index);
    if(itemData == null)
    {
        return null;
    }
    /*get a new item. Every item can use a different prefab,
    the parameter of the NewListViewItem is the prefab' name.
    And all the prefabs should be listed in ItemPrefabList in LoopListView2 Inspector Setting*/
    LoopListViewItem2 item = listView.NewListViewItem("ItemPrefab");
    //get your own component
    BaseVerticalItem itemScript = item.GetComponent<BaseVerticalItem>();
    //IsInitHandlerCalled is false means this item is new created but not fetched from pool.
    if (item.IsInitHandlerCalled == false)
    {
        item.IsInitHandlerCalled = true;
        itemScript.Init();// here to init the item, such as add button click event listener.
    }
    //update the item's content for showing, such as image, text.
    itemScript.SetItemData(itemData, index);
    return item;
}
```

2) NewListViewItem 方法

```
public LoopListViewItem2 NewListViewItem(string itemPrefabName)
```

获取一个新物品, 且该新物品是从名为 **itemPrefabName** 的预制件克隆而来的。

此方法通常用于 **onGetItemByIndex** 中。

3) SetListItemCount 方法

```
public void SetListItemCount(int itemCount, bool resetPos = true)
```

在运行时设置滚动视图的项目总数。如果此参数设置为 -1，则表示项目无限，不支持滚动条，并且项目索引可以从 **-MaxInt** 到 **+MaxInt**。如果此参数设置为大于等于 0 的值，则项目索引只能从 0 到 **itemTotalCount - 1**。如果 resetPos 设置为 false，则此方法执行完毕后，ScrollRect 的内容位置不会改变。

4) 通过项目索引获取显示项目方法

```
public LoopListViewItem2 GetShownItemByItemIndex(int itemIndex)
```

通过 itemIndex 获取可见项。如果该项不可见，则此方法返回 null。

5) 通过商品 ID 获取商品信息方法

```
public LoopListViewItem2 GetShownItemById(int itemId)
```

通过用户定义的 itemId 获取可见项。如果该项不可见，则此方法返回 null。

6) GetShownItemByIndex 方法

```
public LoopListViewItem2 GetShownItemByIndex(int index)
```

通过可见项列表中的索引获取可见项。

所有可见项都存储在一个名为 mItemList 的 List<LoopListViewItem2> 中。

参数索引的范围是从 0 到 mItemList.Count。

7) ShowItemCount 方法

```
public int ShowItemCount
{
    get
    {
        return mItemList.Count;
    }
}
```

获取所有可见项目的总数。

8) 通过项索引刷新项方法

```
public void RefreshItemByItemIndex(int itemIndex)
```

通过 itemIndex 更新一项。如果第 itemIndex 项不可见，则此方法将不执行任何操作。否则，此方法将首先调用 **onGetItemByIndex (itemIndex)** 来获取更新后的项，然后重新定位所有可见项的位置。

9) 刷新所有显示项方法

```
public void RefreshAllShownItem()
```

一次性更新所有可见项目。

10) MovePanelToItemIndex 方法

```
public void MovePanelToItemIndex(int itemIndex, float offset, float duration = 0)
```

此方法将在持续时间（duration）秒内将滚动视图内容的位置移动到（第 itemIndex 项的位置加上偏移量），并且当移动完成时，如果启用了项对齐（ItemSnap），则项对齐也会立即完成。

如果持续时间约为 0，则运动将瞬间完成。

如果持续时间大于 0，则在运动结束时，

‘LoopListView2.mOnSmoothMovePanelToItemFinished’ 委托将会被调用。您可以直接使用如下代码设置 ‘LoopListView2.mOnSmoothMovePanelToItemFinished’：

```
mLoopListView.mOnSmoothMovePanelToItemFinished = YourMovePanelDelegate;
```

关于偏移量参数：

如果偏移量大于 0，则移动结束位置更靠近 ScrollRect 视口；如果偏移量小于 0，则移动结束位置更远离 ScrollRect 视口。

偏移参数的设置范围是从 +（滚动视图大小）到 -（滚动视图大小）。

对于持续时间参数，如果您想要实现近似恒定速度的移动，可以查看 ListViewSimpleMoveDemoScript.cs 中的 OnScrollToButtonClicked 方法。关键代码如下：

```
//get the current first float itemIndex in the ScrollRect viewport.  
float curFloatItemIndex = mLoopListView.GetFirstShownFloatItemIndexInViewPort();  
//get how many items count from current item to the target item.  
float itemCountNeedMove = Mathf.Abs(itemIndex - curFloatItemIndex);  
//get how long time to take.  
float moveTime = itemCountNeedMove * 0.2f;  
//do the movement.  
mLoopListView.MovePanelToItemIndex(itemIndex, 0, moveTime);
```

11) OnItemSizeChanged 方法

```
public void OnItemSizeChanged(int itemIndex)
```

对于垂直滚动的 ScrollRect，当运行时可见项的高度发生变化时，应调用此方法，以使 LoopListView2 组件重新定位所有可见项的位置。对于水平滚动的 ScrollRect，当运行时可见项的宽度发生变化时，应调用此方法，以使 LoopListView2 组件重新定位所有可见项的位置。

12) 立即完成快照法

```
public void FinishSnapImmediately()
```

快照移动将立即完成。

13) CurSnapNearestItemIndex 方法

```
public int CurSnapNearestItemIndex
```

获取视口对齐点附近的项目索引。

14) SetSnapTargetItemIndex 方法

```
public void SetSnapTargetItemIndex(int itemIndex)
```

设置快照目标项的索引。此方法在“PageViewDemo”中使用。

15) ClearSnapData 方法

```
public void ClearSnapData()
```

清除当前的自动对齐目标，然后 LoopScrollView2 将自动对齐到 CurSnapNearestItemIndex。

16) GetFirstShownItemIndexAndOffset 方法

```
public ItemPosStruct GetFirstShownItemIndexAndOffset()
```

此方法可获取列表视图中首个显示项的项索引和偏移量，您可以按如下方式保存返回值：

```
ItemPosStruct ret = mLoopListView.GetFirstShownItemIndexAndOffset();  
ItemPosStruct ret = mLoopListView 获取首个可见项的索引和偏移量
```

如果您想要恢复列表视图的相同位置，可以调用：mLoopListView.MovePanelToItemIndex(ret.mItemIndex, ret.mItemOffset)

17) GetFirstShownFloatItemIndexInViewPort 方法

```
public float GetFirstShownFloatItemIndexInViewPort()
```

获取在滚动矩形视口中首次显示的项目的浮点型项目索引。例如，对于从上到下排列的列表视图，浮点型项目索引 3.25 表示滚动矩形视口中顶部显示项目的项目索引为 3，该项目 25% 的大小在滚动矩形视口之外，75% 的大小在滚动矩形视口之内。

18) 一次性更新项目大小方法

```
public void UpdateItemSizeAtOnce(int startItemIndex, int endItemIndex)
```

如果您能事先知晓每个项目的大小，您可以设置 onGetItemSizeByIndex 委托，该委托用于返回给定 itemIndex 的项目的大小和内边距。

您可以调用 UpdateItemSizeAtOnce 并传入一个项目索引范围，让 LoopListView2 一次性更新这些项目的大小。

例如，当您向 ListView 中添加新项时，如果新项的大小与 LoopListViewInitParam.mItemDefaultWithPaddingSize 不同，并且新项未显示在视口中，那么滚动条的大小将无法正确更新。此时您可以调用 LoopListView2.UpdateItemSizeAtOnce (newItemIndex, newItemIndex) 来让

LoopListView2 会调用您的 **onGetItemSizeByIndex** 方法来获取新添加项的实际大小。

19) 更新列表视图内容方法

```
public void UpdateListViewContent()
```

一次性更新列表视图的内容。您可以调用此方法，让 LoopListView2 立即在 ScrollRect 视口中创建所有项目。

3. 循环网格视图

3.1 循环网格视图概述

LoopGridView 组件附加到了 UGUI ScrollRect 的同一个游戏对象上。

它主要用于 GridView，其中所有项目大小相同，并且行数或列数固定。ScrollRect 可以同时水平和垂直滚动。

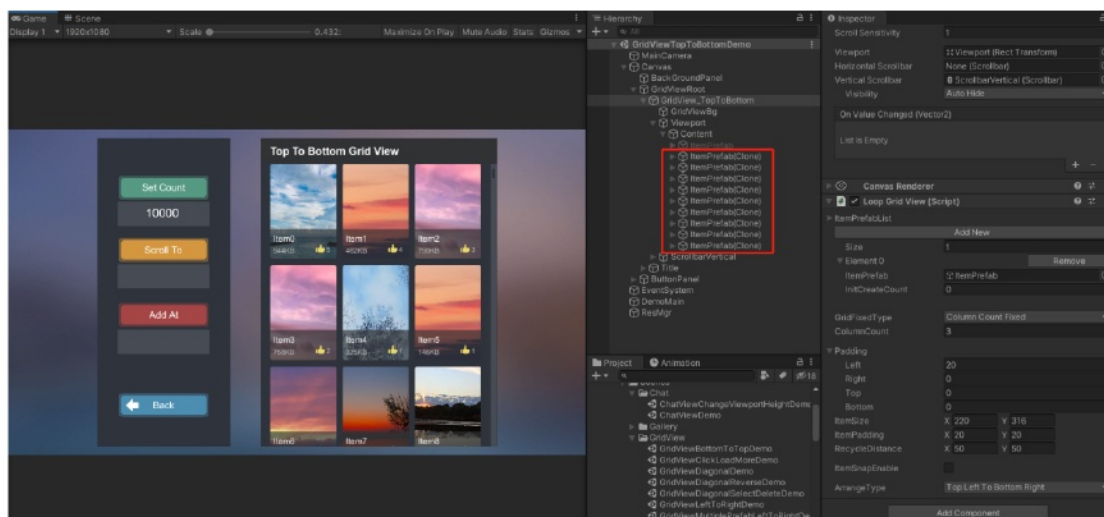
例如，对于一个包含 10,000 项的 GridView，比如 100 行×100 列，LoopGridView 并不会真正创建 10,000 项，而是根据视口的大小创建少量的项。

当滚动视图向上移动时，例如，循环网格视图组件会检查最顶行的位置，一旦最顶行移出视口，循环网格视图组件就会回收最顶行的所有项目。

同时，它会检查最底部行的位置，一旦最底部行接近视口的底部，LoopGridView 组件就会调用 **onGetItemByRowColumn** 处理程序来创建新的一行以及视口内新行中的所有项目，然后将新创建的行置于最底部行的下方，这样新创建的行就成为了新的最底部行。

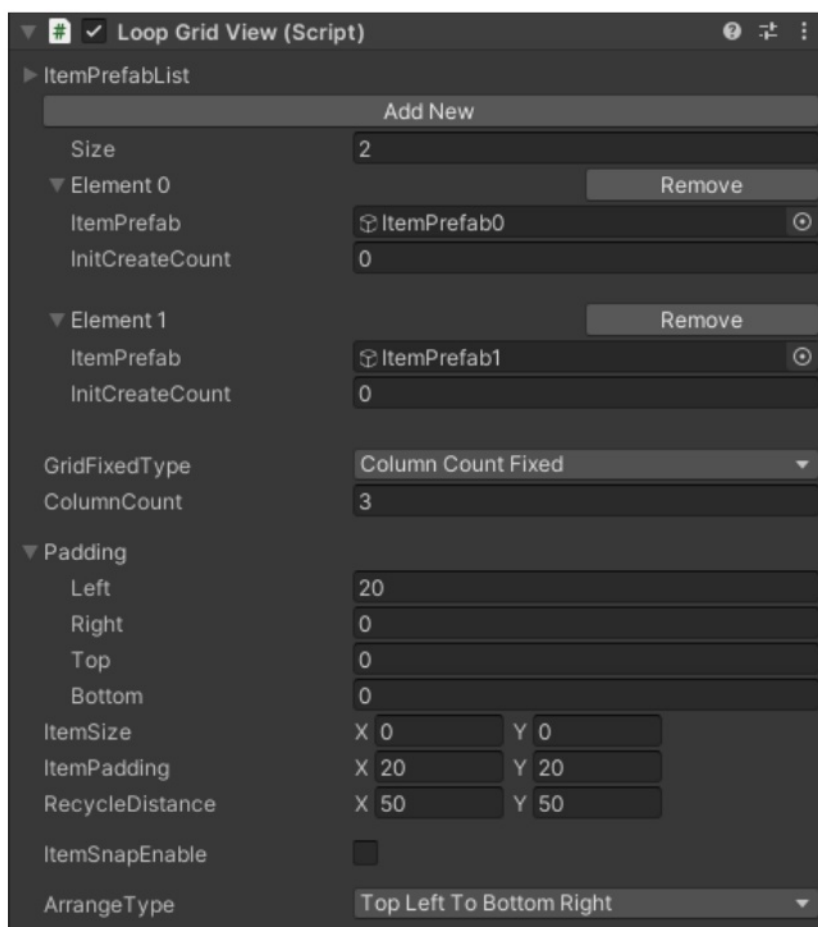
每个物品都可以使用不同的预制件。

GridView 有 10000 个项目，但实际上，只有少数项目真正被创建。下图展示的是一个从左上角到右下角（100 行×100 列）排列的 ScrollRect 的样子：



3.2 循环网格视图检查器设置

在“检查器”中，为了确保 LoopGridView2 组件正常工作，有若干参数需要设置。



1) 物品预制列表

现有的要克隆的项目。

每个物品都可以使用不同的预制件。每个预制件都有一个用于获取和回收操作的池，而 **InitCreateCount** 则是在启动时池中创建的数量。

重要提示：所有 itemPrefab 的本地缩放比例（localScale）都必须为（1,1,1）。并且 ScrollRect 内容游戏对象的本地缩放比例（localScale）也必须为（1,1,1）。

2) 网格固定类型

该值可以是“列数固定”或“行数固定”。此属性类似于 UGUI 的 GridLayoutGroup.constraint 和 GridLayoutGroup.startAxis，即首先应沿哪个轴放置项目。

3) 行数/列数

如果 GridFixedType 为 ColumnCountFixed，则在此处设置 GridView 的列数；
如果 GridFixedType 为 RowCountFixed，则在此处设置 GridView 的行数。此属性类似于 UGUI 的 GridLayoutGroup.constraintCount。

4) 填充

项目与视口之间的间距，此属性类似于 UGUI 的 GridLayoutGroup 的内边距。

5) 物品尺寸

项目大小。如果 x 或 y 被设置为 0，则在 LoopGridView 组件初始化时，**ItemSize** 将自动设置为 **ItemPrefabList** 中第一个项目的大小。

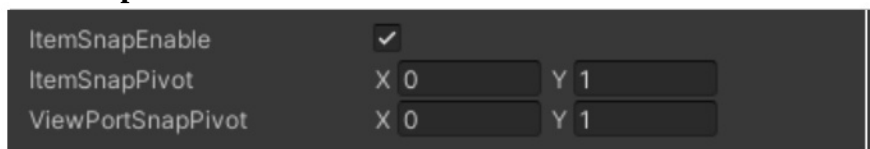
6) 项目内边距

GridView 中每个项目的间距大小。

7) 回收距离

当某个项目即将被回收时，该项目离开视口的距离是多少。

8) ItemSnapEnable：项快启用



如果选中，LoopGridView 组件将尝试将项目对齐到视口中的配置位置。

9) ItemSnapPivot

该项的快照枢轴点的位置，定义为矩形自身大小的分数。（0,0）对应于左下角，而（1,1）对应于右上角。

10) 视口捕捉枢轴

滚动视图（ScrollRect）视口的快照枢轴点的位置，以矩形自身大小的比例来定义。（0,0）对应于左下角，而（1,1）对应于右上角。

11) 排列类型：

滚动方向有四种类型，分别为：从左上到右下、从左下到右上、从右上到左下以及从右下到左上。

3.3 LoopGridView 重要公共方法

1) InitGridView 方法

```
public void InitGridView(int itemTotalCount,
    System.Func<LoopGridView, int, int, int, LoopGridViewItem> onGetItemByRowColumn,
    LoopGridViewSettingParam settingParam = null,
    LoopGridViewInitParam initParam = null)
```

初始化 LoopGridView 组件。有 4 个参数：

项目总数

GridView 中的项目总数。此参数必须设置为 ≥ 0 的值，且 ItemIndex 可以从 0 到 **itemTotalCount - 1**。

获取行列项（onGetItemByRowColumn）

当某个项目进入 ScrollRect 视口时，此操作将被调用，并将项目（itemIndex、行、列）作为参数传递，以便您创建项目并更新其内容。

设置参数

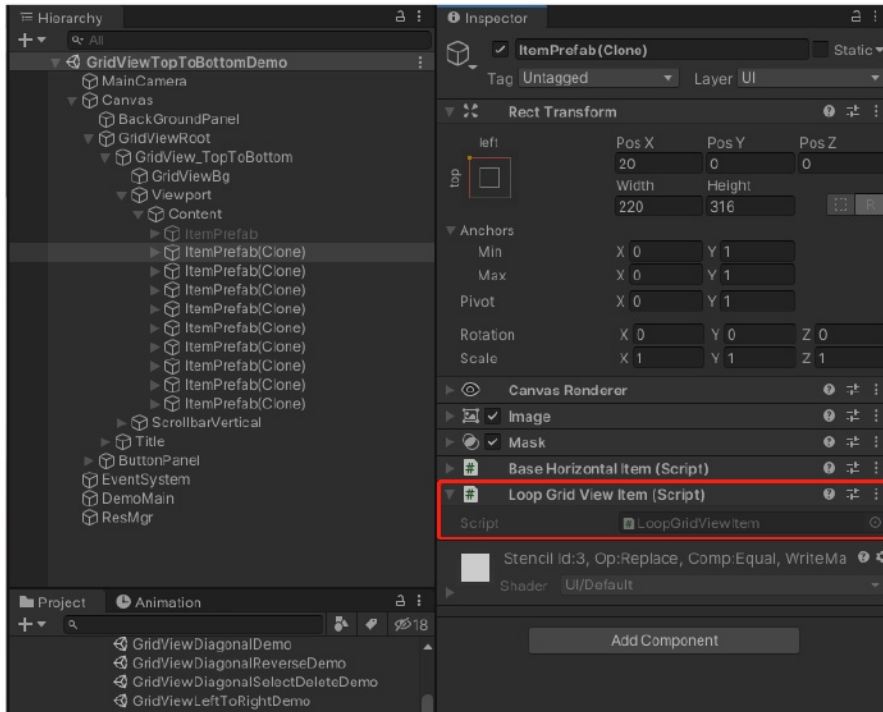
此参数是一个 LoopGridViewSettingParam 对象。您可以使用此参数覆盖检视器设置中的值，例如：

```
LoopGridViewSettingParam settingParam = new LoopGridViewSettingParam();
settingParam.mItemSize = new Vector2(500, 500);
settingParam.mItemPadding = new Vector2(40, 40);
settingParam.mPadding = new RectOffset(10, 20, 30, 40);
settingParam.mGridFixedType = GridFixedType.RowCountFixed;
settingParam.mFixedRowOrColumnCount = 6;
mLoopGridView.InitGridView(DataSourceMgr.Get.TotalItemCount, OnGetItemByIndex, settingParam);
```

循环网格视图项

onGetItemByRowColumn 的返回值。

每个创建的项目都会自动附加一个 LoopGridViewItem 组件。



```
public class LoopGridViewItem : MonoBehaviour
{
    // indicates the item's index in the list the mItemIndex can only be from 0 to
    int mItemIndex = -1;
    // the row index, the item is in. starting from 0.
    int mRow = -1;
    // the column index, the item is in. starting from 0.
    int mColumn = -1;
    //indicates the item's id.
    //This property is set when the item is created or fetched from pool,
    //and will no longer change until the item is recycled back to pool.
    int mItemId = -1;
    LoopGridView mParentGridView = null;
    bool mIsInitHandlerCalled = false;
    string mItemPrefabName;
    RectTransform mCachedRectTransform;
}
```

mItemIndex 属性表示项目的索引。

mItemId 属性表示项目的 ID。此属性在创建项目或从池中获取项目时设置，并且在项目被回收回池之前不会再更改。

以下代码是 **onGetItemByRowColumn** 的一个示例：

```

LoopGridViewItem OnGetItemByRowColumn(LoopGridView gridView, int index, int row, int column)
{
    if (index < 0 || index >= mDataSourceMgr.TotalItemCount)
    {
        return null;
    }
    //get the data to showing
    ItemData itemData = mDataSourceMgr.GetItemDataByIndex(index);
    if (itemData == null)
    {
        return null;
    }
    /*get a new item. Every item can use a different prefab,
    the parameter of the NewListViewItem is the prefab' name.
    And all the prefabs should be listed in ItemPrefabList in LoopGridView Inspector Setting */
    LoopGridViewItem item = gridView.NewListViewItem("ItemPrefab");
    //get your own component
    BaseHorizontalItem itemScript = item.GetComponent<BaseHorizontalItem>();
    // IsInitHandlerCalled is false means this item is new created but not fetched from pool.
    if (item.IsInitHandlerCalled == false)
    {
        item.IsInitHandlerCalled = true;
        itemScript.Init();// here to init the item, such as add button click event listener.
    }
    //update the item' s content for showing, such as image, text.
    itemScript.SetItemData(itemData, index);
    return item;
}

```

2) NewListViewItem 方法

```
public LoopGridViewItem NewListViewItem(string itemPrefabName)
```

获取一个新物品，该新物品是从名为 itemPrefabName 的预制件克隆而来的。此方法通常用于 **onGetItemByRowColumn** 中。

3) SetListItemCount 方法

```
public void SetListItemCount(int itemCount, bool resetPos = true)
```

在运行时设置 GridView 的项目总数。如果 resetPos 为 false，则此方法执行完毕后，ScrollRect 的内容位置不会改变。

4) 通过项目索引获取显示项方法

```
public LoopGridViewItem GetShownItemByItemIndex(int itemIndex)
```

通过 itemIndex 获取可见项。如果该项不可见，则此方法返回 null。

5) 通过项索引获取显示项方法

```
public LoopGridViewItem GetShownItemByRowColumn(int row, int column)
```

通过（行，列）获取可见项。如果该项不可见，则此方法返回空值。

6) 通过项索引刷新项方法

```
public void RefreshItemByItemIndex(int itemIndex)
```

7) 通过 `itemIndex` 更新一项。如果第 `itemIndex` 项不可见，则此方法将不执行任何操作。否则，此方法将调用 `onGetItemByRowColumn` 以获取更新后的项。

8) 通过行和列刷新项方法

```
public void RefreshItemByRowColumn(int row, int column)
```

通过（行，列）更新一个项目。如果该项目不可见，则此方法将不执行任何操作。否则，此方法将调用 `onGetItemByRowColumn` 以获取更新后的项目。

9) 刷新所有已显示项方法

```
public void RefreshAllShownItem()
```

更新所有可见项目。

10) MovePanelToItemByIndex 方法

```
public void MovePanelToItemByIndex(int itemIndex, float offsetX = 0, float offsetY = 0)
```

将滚动视图内容的位置移动到（第 `itemIndex` 项的位置 + 偏移量）。

11) 按行列将移动面板移至项目的方法

```
public void MovePanelToItemByRowColumn(int row, int column, float offsetX = 0, float offsetY = 0)
```

将面板的位置移动到（（行，列）项的位置 + 偏移量）。

12) SetGridFixedGroupCount 方法

```
public void SetGridFixedGroupCount(GridFixedType fixedType, int count)
```

在运行时更改 `GridFixedType` 和固定行/列数。还可以通过 `SetItemSize (Vector2)`、`SetItemPadding (Vector2)`、`SetPadding (RectOffset)` 在运行时更改 `itemSize`、`itemPadding`、`contentPadding`。

13) 立即完成截图方法

```
public void FinishSnapImmediately()
```

快照移动将立即完成。

14) 行列对方法

```
public RowColumnPair CurSnapNearestItemRowColumn
```

获取与视口对齐点最近的项目的行和列。

15) SetSnapTargetItemRowColumn 方法

```
public void SetSnapTargetItemRowColumn(int row, int column)
```

设置快照目标项的行和列。

16) ClearSnapData 方法

```
public void ClearSnapData()
```

清除当前的对齐目标，然后 LoopGridView 将自动对齐到 CurSnapNearestItemRowColumn。

17) ForceUpdateGridViewAtOnce 方法

```
public void ForceUpdateGridViewAtOnce()
```

立即更新网格视图的内容。您可以调用此方法，让 LoopGridView 立即在 ScrollRect 视口中创建所有项目。

4. 循环交错网格视图

4.1 LoopStaggeredGridView 概述

LoopStaggeredGridView 主要用于垂直方向的 GridView（或水平方向的 GridView）中项目高度（或宽度）各不相同的 StaggeredGridView。因此，如果 GridView 中的所有项目大小相同，那么您最好使用 LoopGridView。

LoopStaggeredGridView 与 LoopListView2 类似，对于包含 10,000 个项目的 ScrollRect，LoopStaggeredGridView 并不会真正创建 10,000 个项目，而是根据视口的大小创建少量项目。

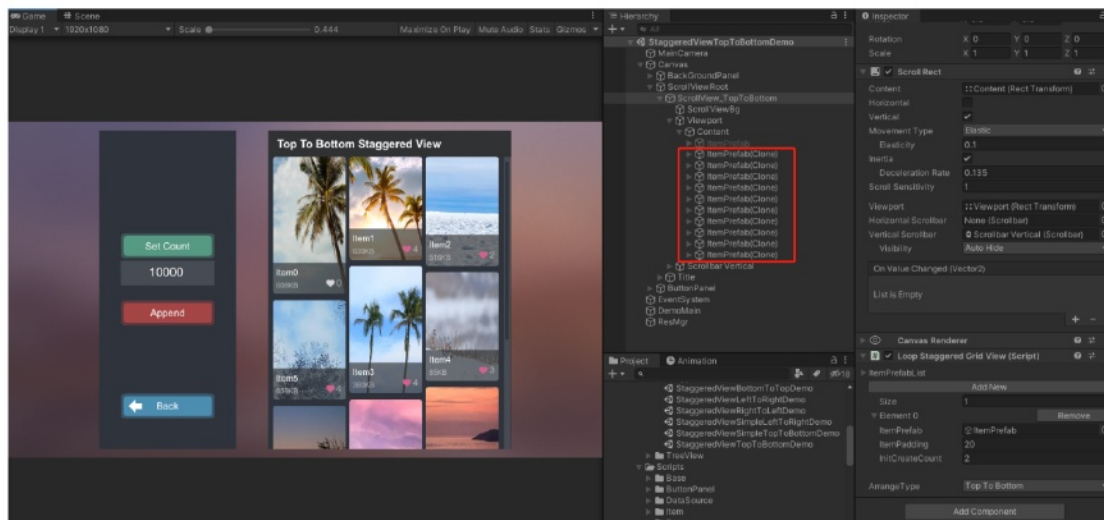
当滚动视图向上滚动时，对于垂直排列的网格视图，LoopStaggeredGridView 组件会检查每一列最顶部项目的所在位置，一旦某列最顶部的项目移出视口，LoopStaggeredGridView 组件就会回收该最顶部的项目。

同时，检查每一列最底部项目的所在位置，一旦某列最底部项目接近视口底部，LoopStaggeredGridView 组件就会调用 `onGetItemByIndex` 处理程序来创建一个新项目，然后将**新建的项目**置于**最短列最底部项目**的下方，即底部位置在所有列中最靠上的那一列。

列的最底部位置。总体而言，所有项目从左至右、从上至下排列。

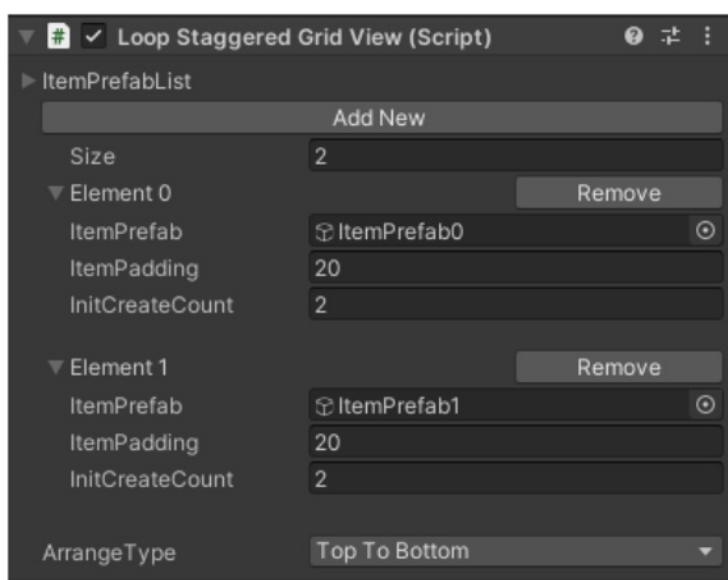
每个项目都可以使用不同的预制件。但对于垂直的 GridView，项目可以有不同的高度，但宽度必须相同。对于水平的 GridView，项目可以有不同的宽度，但高度必须相同。

滚动视图中有 10,000 个项目，但实际上只有少数项目被真正创建。下图展示的是一个顶部到底部排列的 3 列交错网格视图的样子：



4.2 循环交错网格视图检查器设置

在 Inspector 中，为了确保 LoopStaggeredGridView 组件正常工作，有若干参数需要设置。



1) 物品预制列表

现有的要克隆的项目。

每个项目都可以使用不同的预制件，而且每个预制件都可以有不同的**默认内边距**，这是在滚动视图中每个项目之间的间距大小。

每个预制件都有一个用于获取和回收操作的池，而 **InitCreateCount** 则是在启动时池中创建的数量。

实际上，在运行时，每个项目都可以有不同的内边距，您可以在 `onGetItemByIndex` 回调中更改某个项目的内边距。您可以通过 `LoopStaggeredGridViewItem.Padding` 来获取/设置它们。

重要提示：所有 `itemPrefab` 的本地缩放比例都需要为 **(1,1,1)**。

2) 安排类型

滚动方向，这与 `ListView` 相同。

有四种类型：从上到下、从下到上、从左到右、从右到左。

4.3 LoopStaggeredGridView 重要公共方法

1) InitListView 方法

```
public void InitListView(int itemTotalCount, GridViewLayoutParam layoutParam,
    System.Func<LoopStaggeredGridView, int, LoopStaggeredGridViewItem> onGetItemByIndex,
    StaggeredGridViewInitParam initParam = null,
    System.Func<int, (float, float)> onGetItemSizeByItemIndex = null)
```

初始化 `LoopStaggeredGridView` 组件。它有四个参数：

项目总数

滚动视图中的项目总数，此参数应大于等于 0。

```
public class GridViewLayoutParam
{
    public int mColumnOrRowCount = 0;
    public float mItemWidthOrHeight = 0;
    public float mPadding1 = 0;
    public float mPadding2 = 0;
    public float[] mCustomColumnOrRowOffsetArray = null;
```

布局参数

您需要创建一个新的 `GridViewLayoutParam` 实例，并设置您想要的值。

对于垂直的 `GridView`，`mColumnOrRowCount` 表示列数，`mItemWidthOrHeight` 表示项目的宽度，`mPadding1` 表示视口的左边距，`mPadding2` 表示视口的右边距。

对于水平方向的 `GridView`，`mColumnOrRowCount` 表示行数，`mItemWidthOrHeight` 表示项目的高度，`mPadding1` 表示视口的上边距，`mPadding2` 表示视口的下边距。

如果 `mCustomColumnOrRowOffsetArray` 为 `null`，也就是说您未为此参数设置值，那么网格视图将平均排列所有列或行。

如果 `mCustomColumnOrRowOffsetArray` 不为空，则数组中的值为每列/行的 X 偏移量/Y 偏移量，并且 `mCustomColumnOrRowOffsetArray` 的长度必须与 `mColumnOrRowCount` 相同。

获取指定索引项的回调函数 onGetItemByItemIndex

当某个项目进入 ScrollRect 视口时，此操作将被调用，并将项目的索引作为参数传递，以便您创建该项目并更新其内容。

获取指定索引项大小的回调函数

如果您能事先知晓每个项目的大小，您可以设置此委托，该委托用于返回给定 itemIndex 的大小和内边距。

由于每个项目可能具有不同的大小，而 LoopStaggeredGridView 需要项目的大小来进行布局，所以如果您无法事先知道每个项目的大小，LoopStaggeredGridView 的 MovePanelToItemIndex 方法只能移动到滚动视图中曾经显示过的项目。但如果您可以事先知道每个项目的大小，您可以设置此委托，该委托用于返回给定项目索引的大小和内边距，然后当您想要将面板移动到给定的项目索引时，您可以首先调用

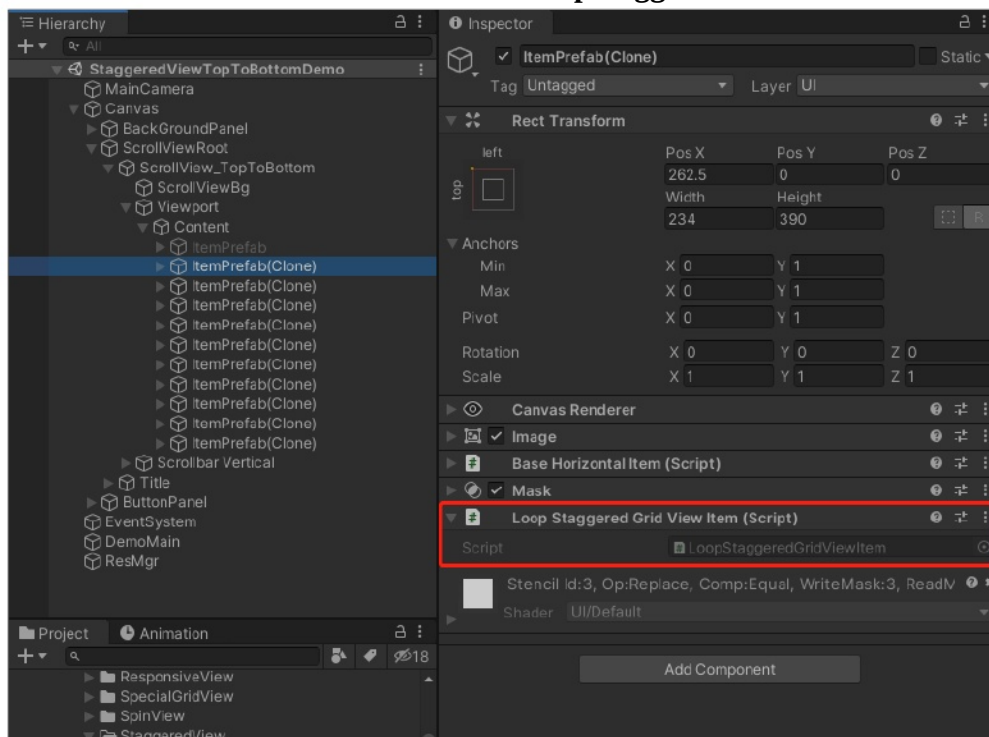
调用 `LoopStaggeredGridView.UpdateContentSizeUpToItemIndex(itemIndex)` 以确保 `LoopStaggeredGridView` 已布局到 `itemIndex` 项，然后调用 `MovePanelToItemIndex(itemIndex)` 即可成功将面板移动到该项。默认情况下，参数为空。

“交错视图移动到项目演示”场景展示了更多细节。

循环交错网格视图项

`onGetItemByItemIndex` 的返回值。

每个创建的项目都会自动附加一个 `LoopStaggeredGridViewItem` 组件。



```
public class LoopStaggeredGridViewItem : MonoBehaviour
{
    int mItemIndex = -1;
    int mItemIndexInGroup = -1;
    int mItemId = -1;
    float mPadding;
    bool mIsInitHandlerCalled = false;
    string mItemPrefabName;
}
```

mItemIndex 属性表示项目的索引。其取值范围是从 0 到 itemTotalCount - 1。

mItemIndexInGroup 属性表示项目在列或行中的索引。对于垂直的 GridView，组表示列；对于水平的 GridView，组表示行。

mItemId 属性表示项目的 ID。此属性在创建项目或从池中获取项目时设置，并且在项目被回收回池之前不会再更改。

以下代码是 **onGetItemByItemIndex** 的一个示例：

```
LoopStaggeredGridViewItem OnGetItemByItemIndex(LoopStaggeredGridView listView, int index)
{
    if (index < 0 || index >= mDataSourceMgr.TotalItemCount)
    {
        return null;
    }
    //get the data to showing
    ItemData itemData = mDataSourceMgr.GetItemDataByIndex(index);
    if(itemData == null)
    {
        return null;
    }
    /*get a new item. Every item can use a different prefab,
    the parameter of the NewListViewItem is the prefab' name.
    And all the prefabs should be listed in ItemPrefabList in LoopStaggeredGridView Inspector Setting */
    LoopStaggeredGridViewItem item = listView.NewListViewItem("ItemPrefab");
    //get your own component
    BaseHorizontalItem itemScript = item.GetComponent<BaseHorizontalItem>();
    // IsInitHandlerCalled is false means this item is new created but not fetched from pool.
    if (item.IsInitHandlerCalled == false)
    {
        item.IsInitHandlerCalled = true;
        itemScript.Init();// here to init the item, such as add button click event listener.
    }
    //update the item' s content for showing, such as image,text.
    itemScript.SetItemData(itemData, index);
    return item;
}
```

2) NewListViewItem 方法

```
public LoopStaggeredGridViewItem NewListViewItem(string itemPrefabName)
```

获取一个新物品，该新物品是从名为 itemPrefabName 的预制件克隆而来的。此方法通常用于 **onGetItemByItemIndex** 中。

3) 设置列表项数量方法

```
public void SetListItemCount(int itemCount, bool resetPos = true)
```

在运行时设置 GridView 的项目总数。如果 resetPos 设置为 false，则

此方法执行完毕后，ScrollRect 的内容位置不会改变。

4) 通过项目索引获取显示项方法

```
public LoopStaggeredGridViewItem GetShownItemByItemIndex(int itemIndex)
```

通过 itemIndex 获取可见项。如果该项不可见，则此方法返回 null。

5) 通过项索引刷新项方法

```
public void RefreshItemByItemIndex(int itemIndex)
```

通过 itemIndex 更新一项。如果第 itemIndex 项不可见，则此方法将不执行任何操作。否则，此方法将首先调用 **onGetItemByItemIndex (itemIndex)** 以获取更新后的项，然后重新定位同一组（即同一列/行）中所有可见项的位置。

6) 刷新所有已显示项方法

```
public void RefreshAllShownItem()
```

更新所有可见项目。

7) MovePanelToItemIndex 方法

```
public void MovePanelToItemIndex(int itemIndex, float offset)
```

将滚动视图内容的位置移动到（第 itemIndex 项的位置 + 偏移量）

8) OnItemSizeChanged 方法

```
public void OnItemSizeChanged(int itemIndex)
```

对于垂直方向的 ScrollRect，当运行时可见项的高度发生变化时，应调用此方法，以使 LoopStaggeredGridView 组件重新定位同一组（即同一列/行）中所有可见项的位置。

对于水平方向的 ScrollRect，当运行时可见项的宽度发生变化时，应调用此方法，以使 LoopStaggeredGridView 组件重新定位同一组（即同一列/行）中所有可见项的位置。